

```
import pandas as pd

df = pd.read_csv("/content/placement_predict_50k Dataset.csv")

print("Dataset loaded successfully!")
print("Shape:", df.shape)

df.head()
```

Dataset loaded successfully!
Shape: (50000, 32)

	StudentID	Gender	City	CollegeTier	Stream	Specialisation	Hostel	HistoryOfBacklogs	SGPA_Sem1	SGPA_Sem2	...
0	1	Male	Ahmedabad	Tier2	ECE	Networking	No	No	6.02	6.54	...
1	2	Female	Mumbai	Tier2	ECE	DataScience	Yes	Yes	5.84	5.12	...
2	3	Male	Kolkata	Tier2	IT	DataScience	Yes	No	4.91	5.29	...
3	4	Male	Jaipur	Tier1	CS	AI	No	No	7.67	8.03	...
4	5	Male	Pune	Tier2	IT	DataScience	Yes	No	8.14	8.97	...

5 rows × 32 columns

```
FEATURES = [
    "CGPA",
    "AttendancePercent",
    "Internships",
    "Projects",
    "Workshops",
    "Certifications",
    "Publications",
    "AptitudeTestScore",
    "SoftSkillsRating",
    "CodingTestScore",
    "MockInterviewScore",
    "ExtraCurricular"
]

TARGET = "Salary Package"

data = df[FEATURES + [TARGET]].copy()

print("Features selected successfully!")
print("Number of features:", len(FEATURES))
print("Target:", TARGET)
print("Data shape:", data.shape)
```

Features selected successfully!
Number of features: 12
Target: Salary Package
Data shape: (50000, 13)

```
data[FEATURES] = data[FEATURES].fillna(
    data[FEATURES].median()
)

print("Missing values handled successfully!")
print("Remaining missing values:", data[FEATURES].isnull().sum().sum())
```

Missing values handled successfully!
Remaining missing values: 0

```
X = data[FEATURES].values
y = data[TARGET].values.reshape(-1, 1)

print("X shape:", X.shape)
print("y shape:", y.shape)
```

X shape: (50000, 12)
y shape: (50000, 1)

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
```

```

    y,
    test_size=0.2,
    random_state=42
)

print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])

```

Training samples: 40000
Testing samples: 10000

```

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

print("Standardization completed!")
print("Training shape:", X_train_s.shape)
print("Testing shape:", X_test_s.shape)

```

Standardization completed!
Training shape: (40000, 12)
Testing shape: (10000, 12)

```

import numpy as np

n_samples, n_features = X_train_s.shape

X_train_b = np.hstack([
    np.ones((n_samples, 1)),
    X_train_s
])

X_test_b = np.hstack([
    np.ones((X_test_s.shape[0], 1)),
    X_test_s
])

print("X_train_b shape:", X_train_b.shape)
print("X_test_b shape:", X_test_b.shape)

```

X_train_b shape: (40000, 13)
X_test_b shape: (10000, 13)

```

def compute_mse(X, y, theta):
    preds = X @ theta
    errors = preds - y
    return np.mean(errors ** 2)

print("MSE function created successfully!")

```

MSE function created successfully!

```

def batch_gradient_descent(X, y, lr=0.05, n_epochs=500):
    m, n = X.shape

    theta = np.zeros((n, 1))

    loss_history = []

    for epoch in range(n_epochs):
        preds = X @ theta
        errors = preds - y

        gradient = (2 / m) * (X.T @ errors)

        theta -= lr * gradient

        loss_history.append(
            compute_mse(X, y, theta)
        )

    return theta, loss_history

print("Batch Gradient Descent function created successfully!")

```

Batch Gradient Descent function created successfully!

```

theta_gd, loss_history_gd = batch_gradient_descent(
    X_train_b,
    y_train,
    lr=0.05,
    n_epochs=500
)

rmse_gd = np.sqrt(
    compute_mse(
        X_test_b,
        y_test,
        theta_gd
    )
)

print(f"[Batch GD] Test RMSE = {rmse_gd:.4f}")

```

[Batch GD] Test RMSE = 4.4105

```

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

print("sklearn libraries imported successfully!")

```

sklearn libraries imported successfully!

```

sk_model = LinearRegression()

sk_model.fit(X_train_s, y_train)

y_pred_sk = sk_model.predict(X_test_s)

rmse_sklearn = np.sqrt(
    mean_squared_error(y_test, y_pred_sk)
)

print(f"[sklearn LR] Test RMSE = {rmse_sklearn:.4f}")

print(
    f"Difference (GD - sklearn) = "
    f"{rmse_gd - rmse_sklearn:.6f}"
)

```

[sklearn LR] Test RMSE = 4.4105
Difference (GD - sklearn) = 0.000000

```

def mini_batch_gradient_descent(
    X,
    y,
    lr=0.05,
    n_epochs=500,
    batch_size=256,
    seed=42
):
    m, n = X.shape

    theta = np.zeros((n, 1))

    loss_history = []

    rng = np.random.default_rng(seed)

    for epoch in range(n_epochs):

        indices = rng.permutation(m)

        X_shuffled = X[indices]
        y_shuffled = y[indices]

        for start in range(0, m, batch_size):

            end = start + batch_size

            X_batch = X_shuffled[start:end]
            y_batch = y_shuffled[start:end]

            preds = X_batch @ theta

            errors = preds - y_batch

            gradient = (

```

```

        2 / X_batch.shape[0]
    ) * (X_batch.T @ errors)

    theta -= lr * gradient

    loss_history.append(
        compute_mse(X, y, theta)
    )

    return theta, loss_history

print("Mini-Batch Gradient Descent function created successfully!")

```

Mini-Batch Gradient Descent function created successfully!

```

theta_mb, loss_history_mb = mini_batch_gradient_descent(
    X_train_b,
    y_train,
    lr=0.05,
    n_epochs=500,
    batch_size=256,
    seed=42
)

rmse_mb = np.sqrt(
    compute_mse(
        X_test_b,
        y_test,
        theta_mb
    )
)

print(f"[Mini-batch GD] Test RMSE = {rmse_mb:.4f}")

```

[Mini-batch GD] Test RMSE = 4.4190

```

import matplotlib.pyplot as plt

print("Matplotlib imported successfully!")

```

Matplotlib imported successfully!

```

plt.figure(figsize=(8, 5))

plt.plot(
    loss_history_gd,
    label="Batch GD"
)

plt.plot(
    loss_history_mb,
    label="Mini-batch GD (batch=256)"
)

plt.xlabel("Epoch")
plt.ylabel("Training MSE")

plt.title(
    "MSE Loss Curve: Batch GD vs Mini-batch GD"
)

plt.legend()

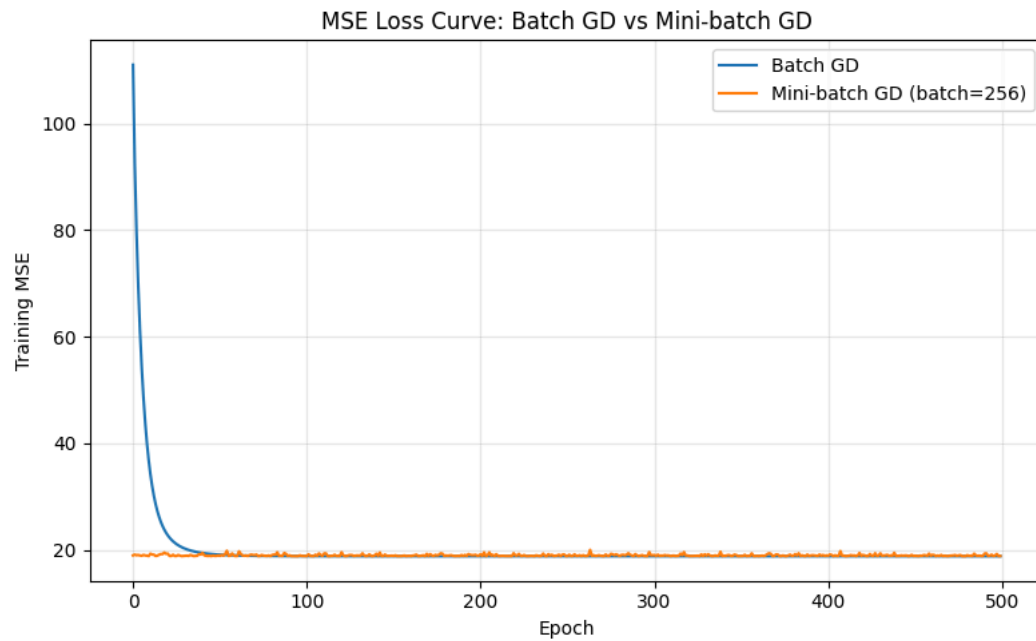
plt.grid(alpha=0.3)

plt.tight_layout()

plt.savefig("loss_curve.png", dpi=150)

plt.show()

```



```
print("\n=== RMSE Comparison ===")

print(f"{'Method':<20}{ 'Test RMSE':>12}")

print(f"{'Batch GD':<20}{rmse_gd:>12.4f}")

print(f"{'Mini-batch GD':<20}{rmse_mb:>12.4f}")

print(f"{'sklearn LinearReg':<20}{rmse_sklearn:>12.4f}")
```

```
=== RMSE Comparison ===
Method          Test RMSE
Batch GD        4.4105
Mini-batch GD   4.4190
sklearn LinearReg 4.4105
```

Start coding or [generate](#) with AI.